Atelier Machine Learning

Analyse Comportementale Clientele Retail

E-commerce de Cadeaux – Pipeline ML Complet

COMPTE RENDU

Rapport technique detaille

Etudiante	Molka Dammak
Module	Machine Learning – GI2 S1
Encadrante	Mme Fadoua Drira
Dataset	E-commerce Retail (4372 clients, 52 features)
Technologies	Python, scikit-learn, Optuna, Flask
Annee	2025–2026

Table des matières

1 Introduction et Contexte **3**
1.1 Contexte du projet . . . 3
1.2 Problematique . . . 3
1.3 Objectifs pedagogiques et techniques . . . 3
1.4 Pipeline de traitement global . . . 4
1.5 Structure du projet . . . 4

2 Description du Dataset **5**
2.1 Vue d'ensemble . . . 5
2.2 Description des variables numeriques principales . . . 5
2.3 Description des variables categorielles . . . 6
2.4 Problemes de qualite identifies . . . 6

3 Pretraitement des Donnees **6**
3.1 Nettoyage des donnees . . . 6
3.1.1 Suppression des colonnes inutiles . . . 6
3.1.2 Extraction de features depuis donnees brutes . . . 7
3.1.3 Traitement des valeurs aberrantes . . . 7
3.2 Separation Train/Test . . . 7
3.3 Gestion des valeurs manquantes . . . 8
3.3.1 Imputation par la mediane – Regle fondamentale . . . 8
3.3.2 KNN Imputer pour Age (30% de NaN) . . . 8
3.4 Feature Engineering . . . 8
3.5 Encodage des variables categorielles . . . 9
3.5.1 Encodage ordinal . . . 9
3.5.2 One-Hot Encoding . . . 9
3.5.3 Target Encoding pour Country . . . 9
3.6 Suppression du Data Leakage . . . 10
3.6.1 Definition . . . 10
3.6.2 Colonnes supprimees . . . 10
3.7 Normalisation et Analyse de la Multicolinearite . . . 10
3.7.1 StandardScaler . . . 10
3.7.2 Analyse VIF . . . 11

4 Modelisation **11**
4.1 Classification – Prediction du Churn . . . 11
4.1.1 Choix du modele : Random Forest Classifier . . . 11
4.1.2 Optimisation des hyperparametres par Optuna . . . 12
4.1.3 Evaluation du modele de classification . . . 13
4.1.4 Importance des features (top 15) . . . 14
4.2 Clustering – Segmentation des Clients . . . 14
4.2.1 Etape 1 : Reduction dimensionnelle par ACP . . . 14
4.2.2 Etape 2 : KMeans k=3 . . . 15
4.2.3 Profils et analyse des 3 segments . . . 15
4.3 Regression – Prediction des Depenses Futures . . . 15
4.3.1 Configuration et preparation . . . 15
4.3.2 Resultats de la regression . . . 16

5 Deploiement – Application Flask **16**
5.1 Architecture de l'API REST 16
5.2 Artefacts charges au demarrage – Pattern Singleton 17
5.3 Pipeline d'inference complet 18

6 Resultats et Analyse **18**
6.1 Resume final des performances 18
6.2 Exemple de predictions sur 10 clients reels 19

7 Conclusion et Perspectives **19**
7.1 Bilan du projet 19
7.2 Points forts 19
7.3 Limites identifiees 20
7.4 Perspectives d'amelioration 20

1 Introduction et Contexte

1.1 Contexte du projet

Dans le domaine du e-commerce, les entreprises accumulent de grandes quantites de donnees transactionnelles sur leurs clients : historique d'achats, frequence de commandes, montants depenses, comportements, satisfaction, etc. Ces donnees constituent une mine d'informations precieuses mais souvent sous-exploitees.

L'entreprise e-commerce de cadeaux au centre de ce projet fait face a un defi majeur : **le churn client**, c'est-a-dire le phenomene par lequel un client cesse d'acheter et quitte definitivement la plateforme. Ce phenomene a des consequences financieres importantes : acquerir un nouveau client coute generalement entre 5 et 25 fois plus cher que de retenir un client existant.

Face a cette problematique, l'entreprise souhaite :

— Mieux **comprendre les profils** de ses clients

— **Anticiper** les departs avant qu'ils ne surviennent

— **Segmenter** sa clientele pour des actions marketing ciblees

— **Estimer** les depenses futures pour prioriser les actions de retention

1.2 Problematique

Question centrale du projet

Comment utiliser les donnees clients pour mieux comprendre leur comportement et predire s'ils vont rester ou partir ?

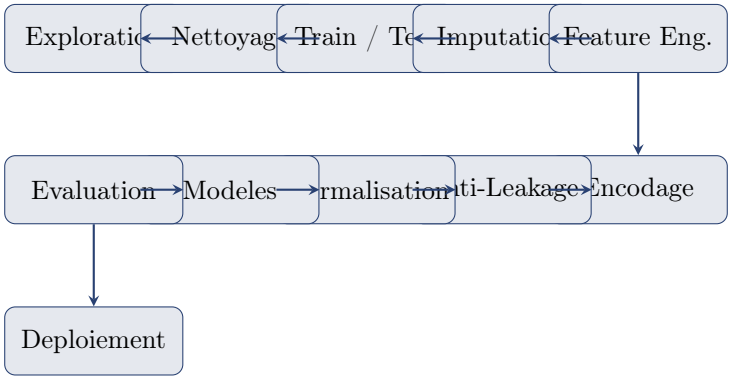
Plus precisement, ce projet repond a trois questions metier :

1. **Ce client va-t-il churner ?** → Classification
2. **A quel segment comportemental appartient-il ?** → Clustering
3. **Combien va-t-il depenser a l'avenir ?** → Regression

1.3 Objectifs pedagogiques et techniques

Competence	Description
Exploration	Analyser la qualite et la structure des donnees
Preparation	Nettoyer, encoder et normaliser les features
Transformation	Reduire la dimension via ACP
Modelisation	Appliquer clustering, classification et regression
Evaluation	Interpreter les resultats et proposer des recommandations
Deploiement	Creer une interface utilisateur avec Flask

1.4 Pipeline de traitement global



1.5 Structure du projet

Arborescence du projet

```
projet_ml_retail/
|-- app/                    # Application Flask
|   |-- app.py              # Serveur Flask + 5 routes API
|   `-- templates/
|       `-- index.html      # Interface utilisateur HTML/JS
|-- data/
|   |-- raw/
|   |   `-- data.csv        # Donnees brutes (4372 clients, 52 features)
|   |-- processed/
|   |   `-- cleaned_data.csv
|   `-- train_test/
|       |-- X_train.csv     # 3497 clients (80%)
|       |-- X_test.csv      #  875 clients (20%)
|       |-- y_train.csv
|       `-- y_test.csv
|-- models/                 # Artefacts ML (.pkl)
|   |-- model.pkl           # RandomForestClassifier
|   |-- scaler.pkl          # StandardScaler classification
|   |-- features.pkl        # 37 features dans l'ordre
|   |-- country_means.pkl   # Target encoding pays
|   |-- impute_values.pkl   # Medianes d'imputation
|   |-- knn_age.pkl         # KNN Imputer pour Age
|   |-- kmeans.pkl          # KMeans k=3
|   |-- pca_cluster.pkl     # PCA 10 composantes
|   |-- reg_model.pkl       # RandomForestRegressor
|   `-- scaler_reg.pkl      # StandardScaler regression
|-- reports/                # Graphiques et rapports
|-- src/                    # Scripts Python
|   |-- preprocessing.py
|   |-- train_model.py
|   |-- predict.py
|   `-- utils.py
|-- requirements.txt
`-- README.md
```

2 Description du Dataset

2.1 Vue d'ensemble

Le dataset est issu d'une plateforme e-commerce de vente de cadeaux au Royaume-Uni. Il contient **4372 clients** decrits par **52 variables** couvrant les dimensions transactionnelles, comportementales et demographiques.

Caracteristique	Total	Train	Test
Nombre de clients	4372	3497 (80%)	875 (20%)
Features brutes	52	–	–
Features apres encodage	–	77	77
Features apres leakage	–	37	37
Taux de churn	33.3%	33.26%	33.26%
Valeurs manquantes totales	1749	–	–
Doublons	0	–	–

2.2 Description des variables numeriques principales

Feature	Intervalle	Description
CustomerID	10000–99999	Identifiant unique client
Frequency	1–50	Nombre de commandes distinctes
MonetaryTotal	-5000 a 15000 £	Somme totale depensee
MonetaryAvg	5–500 £	Montant moyen par commande
TotalQuantity	-10000 a 100000	Quantite totale d'articles
AvgDaysBetweenPurchases	0–365 jours	Delai moyen entre commandes
UniqueProducts	1–1000	Nombre produits differents
TotalTransactions	1–10000	Total lignes transactionnelles
SupportTicketsCount	-1, 0–15, 999	Tickets support (999 = erreur)
SatisfactionScore	-1, 1–5, 99	Note satisfaction (99 = erreur)
Age	18–81 ou NaN	Age estime du client
Churn	0 ou 1	0 = fidele, 1 = parti

2.3 Description des variables categorielles

Feature	Card.	Valeurs	Encodage
Gender	3	M, F, Unknown	One-Hot
AccountStatus	4	Active, Suspended, Pending, Closed	One-Hot
FavoriteSeason	4	Hiver, Printemps, Ete, Automne	One-Hot
Country	37	UK, France, Germany, etc.	Target Enc.
AgeCategory	7	18-24, 25-34, ..., 65+, Inconnu	Ordinal
SpendingCategory	4	Low, Medium, High, VIP	Ordinal
LoyaltyLevel	5	Nouveau, Jeune, Etabli, Ancien, Inconnu	Ordinal
RFMSegment	4	Champions, Fideles, Potentiels, Dormants	One-Hot
CustomerType	5	Hyperactif, Regulier, Occasionnel, ...	One-Hot
Region	8	UK, Europe N/S/E/C, Asie, Autre	One-Hot

2.4 Problemes de qualite identifies

Synthese des problemes qualite

- **Valeurs manquantes :** Age ≈ 30% de NaN, SatisfactionScore et SupportTicketsCount apres correction des aberrantes. Total : **1749 NaN**.
- **Valeurs aberrantes :** SatisfactionScore : -1 et 99 (codes d'erreur systeme). SupportTicketsCount : 999 (code d'erreur). MonetaryTotal : 44 valeurs negatives (retours clients – conservees).
- **Donnees brutes non exploitables :** LastLoginIP (adresse IP unique) et RegistrationDate (formats inconsistants).
- **Desequilibre de classes :** 67% fideles vs 33% churners.
- **Feature inutile :** NewsletterSubscribed = toujours "Yes" – variance nulle.

3 Pretraitement des Donnees

3.1 Nettoyage des donnees

3.1.1 Suppression des colonnes inutiles

Colonne supprimee	Justification
CustomerID	Identifiant unique sans valeur predictive. Si conserve, le modele pourrait memoriser les clients plutot qu'apprendre des patterns generalisables.
NewsletterSubscribed	Valeur constante "Yes" pour 100% des clients. Variance nulle, aucune information discriminante.
LastLoginIP	Adresse IP trop specifique. Remplacee par IP_IsPrivate apres extraction (0 = IP publique, 1 = IP privee).

3.1.2 Extraction de features depuis donnees brutes

```
df['IP_IsPrivate'] = df['LastLoginIP'].str.match(
    r'^(10\.|192\.168\.|172\.(1[6-9]|2[0-9]|3[01])\.)'
).astype(int)
df = df.drop(columns=['LastLoginIP'])
```

Listing 1 – Extraction de IP_IsPrivate

```
df['RegistrationDate'] = pd.to_datetime(
    df['RegistrationDate'],
    format='mixed',
    dayfirst=True,    # Priorite format UK : DD/MM/YYYY
    errors='coerce'   # NaT si format invalide
)
df['RegYear']    = df['RegistrationDate'].dt.year
df['RegMonth']   = df['RegistrationDate'].dt.month
df['RegDay']     = df['RegistrationDate'].dt.day
df['RegWeekday'] = df['RegistrationDate'].dt.weekday
df = df.drop(columns=['RegistrationDate'])
```

Listing 2 – Parsing et decomposition de RegistrationDate

3.1.3 Traitement des valeurs aberrantes

```
# -1 et 99 sont des codes d'erreur systeme, pas de vraies valeurs
df['SatisfactionScore']   = df['SatisfactionScore'].replace([-1, 99], np.nan)
df['SupportTicketsCount'] = df['SupportTicketsCount'].replace([-1, 999], np.nan)
```

Listing 3 – Remplacement des codes d'erreur par NaN

Cas particulier – MonetaryTotal negatif : 44 valeurs negatives ont ete detectees et **conservees deliberement**. Elles representent des retours clients (remboursements) – information comportementale precieuse : un client qui retourne frequemment ses achats est un signal potentiel de churn.

Resultat du nettoyage : 52 colonnes → **53 colonnes** (ajout IP_IsPrivate + 4 features dates, suppression 3 colonnes inutiles)

3.2 Separation Train/Test

La separation est effectuee **immediatement apres le nettoyage**, avant toute imputation ou encodage. Cet ordre est fondamental pour eviter le data leakage.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,    # 20% pour evaluation finale
    random_state=42,  # Graine aleatoire fixee -- reproductibilite
    stratify=y        # Proportion de churn identique dans train et test
)
```

Listing 4 – Separation Train/Test strategifiee

Pourquoi stratify=y ? Sans ce parametre, le split aleatoire pourrait donner 40% de churners dans train et 20% dans test. Le modele apprendrait sur une distribution differente de celle sur laquelle il est evalue, biaisant les resultats.

Ensemble	Taille	% Fideles	% Churners
X_train	3497 (80%)	66.74%	33.26%
X_test	875 (20%)	66.74%	33.26%

3.3 Gestion des valeurs manquantes

3.3.1 Imputation par la mediane – Regle fondamentale

Regle anti-leakage pour l'imputation

La mediane est calculee **uniquement sur X_train**. On applique ensuite la **meme valeur** sur X_train et X_test. Calculer la mediane sur l'ensemble du dataset reviendrait a utiliser des informations du test pendant l'entrainement – c'est du data leakage.

Pourquoi la mediane et non la moyenne ?

Critere	Mediane	Moyenne
Robustesse	Insensible aux outliers	Faussee par les valeurs extremes
Stabilite	Stable sur donnees asymetriques	Instable si distribution biaisee
Adaptation	Adaptee a MonetaryTotal (-5000 a 15000)	Tiree vers les valeurs extremes

```python
numeric_cols = X_train.select_dtypes(include=[np.number]).columns
impute_values = {
    col: float(X_train[col].median())
    for col in numeric_cols
    if X_train[col].notna().any()
}  # 39 colonnes imputees

for col, val in impute_values.items():
    X_train[col] = X_train[col].fillna(val)
    X_test[col]  = X_test[col].fillna(val)

joblib.dump(impute_values, 'models/impute_values.pkl')
```

Listing 5 – Imputation par la mediane du train

3.3.2 KNN Imputer pour Age (30% de NaN)

La variable Age presente environ 30% de valeurs manquantes – le cas le plus critique. Le KNN Imputer est plus adapte car l'age d'un client depend de son profil comportemental global.

```python
from sklearn.impute import KNNImputer

knn = KNNImputer(n_neighbors=5)
# fit_transform sur X_train uniquement
X_train['Age'] = knn.fit_transform(X_train[['Age']]).ravel()
# transform sans re-fit sur X_test
X_test['Age']  = knn.transform(X_test[['Age']]).ravel()
# Sauvegarde pour l'inference Flask
joblib.dump(knn, 'models/knn_age.pkl')
```

Listing 6 – KNN Imputer pour la variable Age

Principe du KNN Imputer : Pour chaque client avec Age manquant, l'algorithme trouve les k=5 clients les plus similaires (en termes de toutes les autres features) et remplace l'Age manquant par la moyenne des ages de ces 5 voisins les plus proches.

3.4 Feature Engineering

```python
# Panier moyen par transaction
# +1 evite la division par zero si Frequency = 0
df['AvgBasketValue']    = df['MonetaryTotal'] / (df['Frequency'] + 1)

# Diversite des produits achetes par transaction
df['DiversityPerTrans'] = df['UniqueProducts'] / (df['Frequency'] + 1)
```

Listing 7 – Creation de nouvelles features comportementales

Feature creee	Formule	Valeur ajoutee
AvgBasketValue	MonetaryTotal / (Frequency + 1)	Distingue un client VIP rare d'un client regulier moyen
DiversityPerTrans	UniqueProducts / (Frequency + 1)	Mesure si le client explore ou achete toujours les memes produits

Resultat : 54 colonnes apres feature engineering.

3.5 Encodage des variables categorielles

3.5.1 Encodage ordinal

Pour les variables avec un **ordre logique**, on attribue un entier croissant afin que le modele comprenne la hierarchie.

```python
ordinal_mappings = {
    'AgeCategory'    : ['18-24','25-34','35-44','45-54','55-64','65+','Inconnu'],
    'SpendingCategory': ['Low','Medium','High','VIP'],
    'LoyaltyLevel'   : ['Nouveau','Jeune','Etabli','Ancien','Inconnu'],
}
for col, order in ordinal_mappings.items():
    if col in df.columns:
        mapping = {v: i for i, v in enumerate(order)}
        df[col] = df[col].map(mapping).fillna(-1).astype(int)
```

Listing 8 – Encodage ordinal avec ordre defini

Exemple AgeCategory : 18-24 $\rightarrow$ 0 | 25-34 $\rightarrow$ 1 | 35-44 $\rightarrow$ 2 | ... | 65+ $\rightarrow$ 5

3.5.2 One-Hot Encoding

Pour les variables **sans relation d'ordre**, une colonne binaire est creee par categorie. Si on utilisait l'ordinal, le modele croirait a tort que "Automne" est deux fois plus grand que "Printemps".

```python
onehot_cols = ['Gender','FavoriteSeason','AccountStatus',
               'Region','RFMSegment','CustomerType',
               'WeekendPreference','ProductDiversity']
df = pd.get_dummies(df, columns=onehot_cols, drop_first=True)
# drop_first=True evite la multicolinearite parfaite
```

Listing 9 – One-Hot Encoding

Exemple Gender : M $\rightarrow$ Gender_M = 1, Gender_F = 0

3.5.3 Target Encoding pour Country

Avec 37+ pays, le One-Hot creerait 37 colonnes binaires – explosion de dimension et multicolinearite. Le Target Encoding remplace chaque pays par son taux de churn moyen, calcule **uniquement sur X_train**.

```python
def fit_country_encoder(X_train, y_train):
    temp = X_train[['Country']].copy()
    temp['Churn'] = y_train.values
    return temp.groupby('Country')['Churn'].mean().to_dict()

country_means = fit_country_encoder(X_train, y_train)
# Exemple : United Kingdom -> 0.23 (23% de churners parmi les clients UK)
df['Country_encoded'] = df['Country'].map(country_means)
df = df.drop(columns=['Country'])
```

Listing 10 – Target Encoding pour Country

Resultat : 54 → **77 colonnes** apres encodage complet.

3.6 Suppression du Data Leakage

3.6.1 Definition

Data Leakage – Definition

Le data leakage survient quand le modele utilise des informations qui ne seraient **pas disponibles au moment de la prediction reelle**. Les resultats semblent excellents a l'entrainement mais le modele est inutilisable en production.
Exemple : Si on conserve ChurnRiskCategory = "Critique", le modele predit churn = 1 avec 99% d'accuracy – mais uniquement parce qu'il lit directement la reponse. En production, ce champ n'existe pas encore.

3.6.2 Colonnes supprimees

40 colonnes supprimees sur 77 :

Categorie	Colonnes
Derivees du churn (fuite directe)	ChurnRiskCategory, LoyaltyLevel, SpendingCategory, RFMSegment_Dormants, RFMSegment_Fideles, RFMSegment_Potentiels
Posterieures a l'evenement	Recency, CustomerTenureDays, FirstPurchaseDaysAgo, AccountStatus_Closed, AccountStatus_Pending, AccountStatus_Suspended
One-Hot redondants	CustomerType (4 modalites), FavoriteSeason (3 modalites), Gender_M, Gender_Unknown, WeekendPreference (2 modalites), ProductDiversity (2 modalites), Region (toutes), PreferredMonth, UniqueDescriptions, UniqueInvoices

Resultat : 77 colonnes → **37 features propres**.

3.7 Normalisation et Analyse de la Multicolinearite

3.7.1 StandardScaler

```python
scaler = StandardScaler()
X_train_s = pd.DataFrame(
    scaler.fit_transform(X_train),  # fit + transform sur train uniquement
    columns=X_train.columns
```

```
)
X_test_s = pd.DataFrame(
    scaler.transform(X_test),        # transform seulement sur test
    columns=X_test.columns
)
joblib.dump(scaler, 'models/scaler.pkl')
```

Listing 11 – Normalisation StandardScaler

Formule : $z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$
Apres transformation : moyenne = 0, variance = 1 pour toutes les features.
Pourquoi normaliser ?

— KMeans calcule des distances euclidiennes – sans normalisation, MonetaryTotal (jusqu'a 15000) dominerait Age (18-81).

— PCA est sensible a la variance – les features a grande echelle domineraient les axes principaux.

3.7.2 Analyse VIF

Feature	VIF	Interpretation
NegativeQuantityCount	$+\infty$	Correlation parfaite – geree par anti-leakage
CancelledTransactions	$+\infty$	Correlation parfaite – geree par anti-leakage
RegYear	840.19	Tres eleve, feature temporelle utile conservee
UniqueCountries	597.83	Eleve, info geographique conservee
MonetaryMin / Max	419 / 417	Correles a MonetaryTotal – conserves
Age	21.88	Acceptable apres KNN imputation

Interpretation : VIF < 5 = acceptable | VIF > 10 = problematique | VIF $= \infty$ = correlation parfaite.

4 Modelisation

4.1 Classification – Prediction du Churn

4.1.1 Choix du modele : Random Forest Classifier

Algorithme	Avantages	Inconvenients	Adapte ?
Random Forest	Robuste, interpretable, gere le desequilibre	Plus lent qu'un arbre simple	**Oui**
Reg. logistique	Rapide, interpretable	Assume la linearite	Moyen
SVM	Performant en haute dim.	Lent, peu interpretable	Moyen
XGBoost	Tres performant	Plus complexe a tuner	Oui

Justifications du choix Random Forest :

— **Robustesse :** Ensemble de 155–300 arbres – resiste au surapprentissage.

— **Desequilibre :** `class_weight='balanced'` pondere automatiquement les churners.

— **Interpretabilite :** Fournit l'importance de chaque feature.
— **Flexibilite :** Gere naturellement les donnees mixtes.
— **Non-linearite :** Capture les interactions complexes entre variables.

4.1.2 Optimisation des hyperparametres par Optuna

Optuna est un framework d'optimisation bayesienne plus intelligent que GridSearchCV car il utilise les resultats des essais precedents pour guider la recherche vers les zones les plus prometteuses de l'espace des hyperparametres.

```
import optuna

def objective(trial):
    params = {
        'n_estimators'     : trial.suggest_int('n_estimators', 50, 300),
        'max_depth'        : trial.suggest_int('max_depth', 5, 20),
        'min_samples_split': trial.suggest_int('min_samples_split', 2, 10),
        'min_samples_leaf' : trial.suggest_int('min_samples_leaf', 1, 5),
        'max_features'     : trial.suggest_categorical('max_features',
                                                        ['sqrt', 'log2']),
        'class_weight'     : 'balanced',
        'random_state'     : 42,
    }
    model = RandomForestClassifier(**params)
    # Score objectif : F1, pas accuracy -- adapte aux classes desequilibrees
    return cross_val_score(model, X_train_s, y_train,
                           cv=3, scoring='f1').mean()

# Seed fixe pour reproductibilite totale des resultats
sampler = optuna.samplers.TPESampler(seed=42)
study   = optuna.create_study(direction='maximize', sampler=sampler)
study.optimize(objective, n_trials=20, show_progress_bar=True)
```

Listing 12 – Optimisation Optuna avec seed fixe

Pourquoi F1 comme score objectif et non l'Accuracy ?

Avec 67% de fideles et 33% de churners, un modele qui predit systematiquement "fidele" obtiendrait 67% d'accuracy sans rien apprendre. Le F1 equilibre precision et recall :

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Meilleurs hyperparametres trouves :

Hyperparametre	Valeur	Role
n_estimators	211	Nombre d'arbres dans la foret
max_depth	16	Profondeur maximale de chaque arbre
min_samples_split	6	Min. d'echantillons pour diviser un noeud
min_samples_leaf	4	Min. d'echantillons dans une feuille
max_features	sqrt	Features considerees a chaque split
class_weight	balanced	Poids inverse des frequences de classe
random_state	42	Reproductibilite garantie

4.1.3 Evaluation du modele de classification

Metriques sur X_test (875 clients jamais vus) :

Metrique	Valeur	Seuil excellent	Evaluation
Accuracy	**80.6%**	$> 80\%$	Excellent
AUC-ROC	**0.856**	> 0.80	Tres bon
F1 Churn	**0.70**	> 0.70	Bon
F1 CV-5	**0.683**	> 0.65	Bon
Stabilite	**±0.005**	$< \pm 0.02$	Tres stable

Interpretation de l'AUC-ROC = 0.856 : Si on prend au hasard un churner et un fidele, le modele classe correctement le churner avant le fidele dans **85.6% des cas** – mesure de discrimination independante du seuil de decision.

Matrice de confusion :

	Predit : Fidele (0)	Predit : Churn (1)
Reel : Fidele (0)	TN = 509	FP = 75
Reel : Churn (1)	FN = 95	TP = 196

- **509 fideles** correctement identifies (vrais negatifs)
- **196 churners** correctement detectes (vrais positifs)
- **75 fausses alarmes :** fideles predits comme churners (cout : offres inutiles)
- **95 churners manques :** clients partis non detectes (cout : perte de revenus)

Rapport de classification complet :

Classe	Precision	Recall	F1	Support
Fidele (0)	84%	87%	86%	584
Churn (1)	72%	67%	70%	291
Accuracy	–	–	**80.6%**	875
Macro avg	78%	77%	78%	875
Weighted avg	80%	80%	80%	875

Validation croisee 5 folds :

Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Moyenne	Ecart-type
0.691	0.685	0.679	0.686	0.677	**0.683**	**±0.005**

L'ecart-type de **±0.005** confirme l'absence totale d'overfitting et une generalisation stable sur tous les folds.

4.1.4 Importance des features (top 15)

Feature	Importance	Interpretation metier
TotalTransactions	8.74%	Engagement total du client
Frequency	7.92%	Regularite des achats
AvgDaysBetweenPurchases	7.30%	Rythme d'achat
TotalQuantity	7.16%	Volume d'achats
MonetaryTotal	6.95%	Valeur financiere du client
UniqueProducts	5.79%	Diversite des interets
RegYear	5.48%	Anciennete (annee d'inscription)
RegMonth	5.02%	Saisonnalite d'inscription
AvgBasketValue	3.74%	Valeur moyenne par transaction
RegDay	3.39%	Jour d'inscription
MonetaryMin	3.15%	Depense minimale enregistree
AvgQuantityPerTransaction	2.90%	Quantite moyenne par commande
MonetaryStd	2.75%	Variabilite des depenses
MonetaryAvg	2.73%	Depense moyenne par commande
AvgLinesPerInvoice	2.66%	Complexite des factures

Les variables **RFM** (Frequency, Monetary) dominent – coherent avec la litterature marketing sur la prediction du churn.

4.2 Clustering – Segmentation des Clients

4.2.1 Etape 1 : Reduction dimensionnelle par ACP

Pourquoi l'ACP avant KMeans ?

En haute dimension (37 features), les distances euclidiennes perdent leur sens discriminant – c'est la **malediction de la dimensionnalite**. Tous les points tendent a devenir equidistants, rendant le clustering inefficace.

```
from sklearn.decomposition import PCA

pca_cluster  = PCA(n_components=10, random_state=42)
X_train_pca  = pca_cluster.fit_transform(X_train_s)
var_explained = pca_cluster.explained_variance_ratio_.sum()
# var_explained = 66.9%
```

Listing 13 – PCA avant clustering

Composantes	Variance	Choix ?	Justification
10	66.9%	✓ Retenu	Compacite + efficacite
15	80.0%	–	Trop de composantes
19	90.0%	–	Trop de composantes
23	95.0%	–	Trop de composantes

Suppression des outliers extremes avant le fit :

```
norms = np.linalg.norm(X_train_pca, axis=1)
Q1, Q3 = np.percentile(norms, 25), np.percentile(norms, 75)
IQR    = Q3 - Q1
# Seuil de Tukey : Q3 + 1.5 x IQR
mask   = norms <= (Q3 + 1.5 * IQR)
n_outliers = (~mask).sum()  # 209 outliers retires
X_train_pca_clean = X_train_pca[mask]
```

Listing 14 – Suppression des outliers avant KMeans

4.2.2 Etape 2 : KMeans k=3

```
# Methode du coude : k teste de 2 a 8
inertias = []
for k in range(2, 9):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_train_pca_clean)
    inertias.append(km.inertia_)
# k=3 retenu : point ou l'inertie diminue le moins vite

# Modele final
kmeans   = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(X_train_pca_clean)          # fit sur donnees propres
clusters = kmeans.predict(X_train_pca) # predict sur tous les points
```

Listing 15 – KMeans avec methode du coude

4.2.3 Profils et analyse des 3 segments

Cluster	Clients	%	Churn	Freq.	Monetary
0 – Actifs standard	1555	44.5%	35.1%	+0.049	+0.055
1 – Faible valeur	1132	32.4%	**36.1%**	-0.093	-0.105
2 – Meilleurs clients	810	23.2%	**25.7%**	+0.037	+0.042

Frequency et Monetary sont en valeurs normalisees (ecarts-types par rapport a la moyenne).

Analyse detaillee des 3 segments

- **Cluster 0 – Actifs standard (44.5%) :** Clients avec frequence et depense legerement au-dessus de la moyenne. Churn modere a 35.1%. *Action : Fidelisation ciblee par des programmes de recompenses.*
- **Cluster 1 – Faible valeur (32.4%) :** Clients avec frequence et depense inferieures a la moyenne. Churn le plus eleve a 36.1%. *Action : Retention urgente avec offres promotionnelles ciblees.*
- **Cluster 2 – Meilleurs clients (23.2%) :** AvgDays tres negatif (-0.222) indique des achats tres frequents et groupes. Churn le plus bas a 25.7%. *Action : Reactiver en priorite si des signes de churn sont detectes.*

Validation naturelle : Le clustering est **non supervise** – il ne voit pas le churn pendant l'entrainement. Pourtant, il identifie naturellement les meilleurs clients (Cluster 2) avec le churn le plus bas. Cela valide la qualite du clustering.

4.3 Regression – Prediction des Depenses Futures

4.3.1 Configuration et preparation

```
REG_FEATURES = [
    'Frequency', 'TotalQuantity', 'Age', 'UniqueProducts',
    'SatisfactionScore', 'SupportTicketsCount',
    'TotalTransactions', 'AvgDaysBetweenPurchases'
]
REG_TARGET = 'MonetaryTotal'

# Filtrage outliers MonetaryTotal (percentile 1% - 99%)
Q1_r, Q3_r = df_reg[REG_TARGET].quantile([0.01, 0.99])
df_reg = df_reg[
```

```
    (df_reg[REG_TARGET] >= Q1_r) &
    (df_reg[REG_TARGET] <= Q3_r)
]
# 4284 lignes conservees sur 4372
```

Listing 16 – Preparation des donnees pour la regression

```
scaler_reg  = StandardScaler()
X_reg_tr_s  = scaler_reg.fit_transform(X_reg_tr)
X_reg_te_s  = scaler_reg.transform(X_reg_te)

reg_model = RandomForestRegressor(
    n_estimators=200, random_state=42, n_jobs=-1
)
reg_model.fit(X_reg_tr_s, y_reg_tr)

y_reg_pred = reg_model.predict(X_reg_te_s)
# Contrainte metier : la depense est toujours positive
y_reg_pred = np.maximum(0, y_reg_pred)
```

Listing 17 – Entrainement du modele de regression

4.3.2 Resultats de la regression

Metrique	Valeur	% erreur	Seuil bon	Evaluation
MAE	375.61 £	**28.2%**	$< 30\%$	Bon
RMSE	772.75 £	**57.9%**	$< 50\%$	Acceptable
R^2	**0.8385**	**84% explique**	> 0.80	Tres bon

Depense moyenne du dataset test : 1333.87 £

Interpretation des metriques :

- **MAE = 375£ (28.2%) :** En moyenne, le modele se trompe de ±375£. Sur une plage de -5000 a +15000£, c'est acceptable.
- **RMSE = 773£ (57.9%) :** Plus eleve que la MAE car le RMSE penalise les grandes erreurs au carre. Les clients VIP (comme le client 12748 qui depense 14 933£) creent des erreurs ponctuelles importantes. C'est inevitable sur des donnees retail heterogenes.
- **R^2 = 0.839 :** Le modele explique **84% de la variance** des depenses. C'est le critere principal – excellent pour des donnees e-commerce tres bruitees.

5 Deploiement – Application Flask

5.1 Architecture de l'API REST

Pourquoi Flask ?

Flask est un micro-framework Python leger et flexible, ideal pour creer des API REST servant des modeles ML. Il permet d'exposer nos modeles via des routes HTTP simples et d'integrer facilement une interface web sans complexite excessive.

L'application expose **5 routes** :

Route	Methode	Fonctionnement
/	GET	Affiche l'interface HTML
/health	GET	Verifie que l'API fonctionne
/random_client	GET	Pioche un client aleatoire depuis data.csv
/predict	POST	Recoit JSON client, retourne toutes les predictions
/verify/{id}	GET	Compare la prediction au vrai churn (validation)

5.2 Artefacts charges au demarrage – Pattern Singleton

Les 10 artefacts sont charges **une seule fois au demarrage**. Sans ce pattern, chaque requete chargerait 10 fichiers pkl, ajoutant plusieurs secondes de latence.

```
_model = None  # Variable globale -- None = pas encore charge

def load_artifacts():
    global _model, _scaler, _knn_age, _kmeans, _pca_cluster
    global _reg_model, _scaler_reg, _reg_features
    global _feature_names, _country_means, _impute_values

    if _model is not None:  # Deja charge -- ne recharge pas
        return

    _model         = joblib.load(MODEL_PATH)
    _scaler        = joblib.load(SCALER_PATH)
    _knn_age       = joblib.load(KNN_AGE_PATH)
    _kmeans        = joblib.load(KMEANS_PATH)
    _pca_cluster   = joblib.load(PCA_CLUSTER_PATH)
    _reg_model     = joblib.load(REG_MODEL_PATH)
    _scaler_reg    = joblib.load(SCALER_REG_PATH)
    _feature_names = joblib.load(FEATURES_PATH)
    _country_means = joblib.load(CM_PATH)
    _impute_values = joblib.load(IMPUTE_PATH)
    _reg_features  = joblib.load(REG_FEATURES_PATH)
```

Listing 18 – Pattern Singleton pour le chargement des artefacts

Artefact	Role
model.pkl	RandomForestClassifier entraine
scaler.pkl	StandardScaler pour la classification
features.pkl	Liste ordonnee des 37 features
country_means.pkl	Target encoding des 37+ pays
impute_values.pkl	Medianes pour l'imputation (39 colonnes)
knn_age.pkl	KNN Imputer pour la variable Age
kmeans.pkl	KMeans k=3 pour le clustering
pca_cluster.pkl	PCA 10 composantes pour le clustering
reg_model.pkl	RandomForestRegressor
scaler_reg.pkl	StandardScaler pour la regression

5.3 Pipeline d'inference complet

Les 6 etapes du pipeline d'inference – /predict

1. **Reception JSON** → `transform_for_inference()` applique exactement le meme preprocessing qu'a l'entrainement (nettoyage, KNN Age, feature engineering, encodage).
2. **Normalisation :** `scaler.transform()` – StandardScaler fitte sur X_train.
3. **Classification :** `model.predict()` + `predict_proba()` → churn_pred (0/1) + churn_proba (0.0–1.0) + risk_level (Faible / Moyen / Eleve / Critique).
4. **Clustering :** `pca_cluster.transform()` + `kmeans.predict()` → cluster (0/1/2) + cluster_label.
5. **Regression :** `scaler_reg.transform()` + `reg_model.predict()` → monetary_pred (depense estimee en £).
6. **Reponse JSON** contenant les 3 resultats en une seule requete.

```python
def risk_label(p: float) -> str:
    if   p < 0.20: return "Faible"
    elif p < 0.40: return "Moyen"
    elif p < 0.70: return "Eleve"
    else:          return "Critique"
```

Listing 19 – Niveaux de risque bases sur la probabilite de churn

6 Resultats et Analyse

6.1 Resume final des performances

Metrique	Valeur finale	Seuil excellent	Evaluation
Classification (Random Forest + Optuna)			
Accuracy	**80.6%**	> 80%	Excellent
AUC-ROC	**0.856**	> 0.80	Tres bon
F1 Churn	**0.70**	> 0.70	Bon
F1 CV-5	**0.683**	> 0.65	Bon
Stabilite	**±0.005**	< ±0.02	Tres stable
Clustering (PCA + KMeans)			
Segments identifies	**3 distincts**	–	Coherents
Variance PCA	**66.9%**	–	Suffisant
Ecart churn clusters	**10.4%**	> 10%	Discriminant
Regression (Random Forest Regressor)			
R^2	**0.839**	> 0.80	Tres bon
MAE	**375£ (28.2%)**	< 30%	Bon
RMSE	**773£ (57.9%)**	< 50%	Acceptable

6.2 Exemple de predictions sur 10 clients reels

CustomerII	Churn	Probabilite	Risque	Cluster	Depense prevue
15484	0	11.65%	Faible	2	4 060.08 £
16952	1	87.00%	Critique	0	205.52 £
15745	0	8.74%	Faible	0	1 014.53 £
13570	1	63.08%	Eleve	2	250.78 £
13747	1	95.42%	Critique	0	76.20 £
12604	0	26.99%	Moyen	0	301.33 £
15054	0	15.80%	Faible	2	331.23 £
12999	1	66.92%	Eleve	0	503.73 £
13149	0	13.87%	Faible	0	2 155.41 £
12748	0	2.69%	Faible	2	14 933.62 £
Resume : 4/10 churners predits | Probabilite moy. : 39.28% | Depense moy. : 2 383.24 £					

Observations cles :

- Le client **13747** a une probabilite de churn de **95.42%** mais une depense prevue de seulement 76£– faible priorite de retention (cout d'action superieur au revenu espere).
- Le client **12748** est fidele (2.69% de churn) avec une depense prevue de **14 933£** – client VIP a conserver absolument.
- Le client **16952** a une probabilite de churn de 87% et une depense de 205£– action de retention ciblee recommandee.

7 Conclusion et Perspectives

7.1 Bilan du projet

Ce projet a permis de developper une solution Machine Learning complete couvrant l'integralite de la chaine de traitement data science, depuis l'exploration des donnees brutes jusqu'au deploiement d'une API Flask fonctionnelle avec interface utilisateur.

7.2 Points forts

- **Pipeline ML rigoureux :** Preprocessing → Modelisation → Deploiement sans data leakage, avec reproductibilite garantie (seed=42 partout).
- **Approche multi-modeles complementaire :** Classification (churn), Clustering (segmentation), Regression (LTV) – les 3 modeles se completent pour une vision a 360 degres du client.
- **Imputation avancee :** KNN Imputer pour Age (30% de NaN), mediane robuste pour les autres variables numeriques (39 colonnes imputees).
- **Gestion rigoureuse du data leakage :** 40 colonnes supprimees pour garantir des predictions realistes et utilisables en production. Sans cette etape, l'accuracy artificielle aurait ete de 99% – inutilisable en reel.
- **Resultats solides :** AUC-ROC = 0.856 et R^2 = 0.839 – performances au-dessus des seuils industriels "tres bon".
- **Stabilite exceptionnelle :** F1 CV-5 = 0.683 ± 0.005 – absence totale d'overfitting confirmee.
- **Deploiement complet :** API Flask avec 5 routes, 10 artefacts, interface HTML interactive et pipeline d'inference en 6 etapes.

7.3 Limites identifiees

- **F1 Churn = 0.70 :** Le modele rate encore 95 churners sur 291 (faux negatifs). Ces clients partent sans qu'on ait pu intervenir.
- **RMSE = 57.9% :** Les clients VIP a tres forte depense creent des erreurs importantes dans la regression. Un modele specifique VIP pourrait etre envisage.
- **Dataset limite :** 4372 clients est suffisant pour Random Forest mais insuffisant pour des approches Deep Learning plus complexes.
- **Multicolinearite residuelle :** Certaines features conservent un VIF eleve (RegYear = 840). Une reduction complementaire aurait pu ameliorer la stabilite.

7.4 Perspectives d'amelioration

Amelioration	Impact attendu
SMOTE	Generer des exemples synthetiques de churners pour ameliorer F1 Churn (gain estime : +5 a 10%).
XGBoost / LightGBM	Algorithmes souvent plus performants sur donnees tabulaires.
Ajustement du seuil	Passer de 0.5 a 0.4 pour reduire les faux negatifs (churners non detectes).
Enrichissement donnees	Ajouter des features temporelles pour capturer l'evolution du comportement client dans le temps.
Deep Learning	Explorer les reseaux de neurones avec un dataset plus grand (> 50k clients).
Monitoring en prod.	Detecter le data drift et reentrainer periodiquement.
Deploiement cloud	Migrer l'API Flask vers AWS/Azure pour la scalabilite.

Resume Final du Projet

Ce projet demontre la faisabilite et l'efficacite d'une approche Machine Learning complete pour l'analyse comportementale de clients e-commerce.

80.6%	**0.856**	**0.839**	**3 segments**
Accuracy	AUC-ROC	R^2 Regression	Clustering

Pipeline ML complet – 10 artefacts – API Flask – Predictions en temps reel